Bayesian Inference for Predicting K% in the MLB

Lindsay Morris

March 28, 2026

This model aims to predict an MLB pitcher's strikeout percentage in the 2025 season given pitch data from 2021 through 2024.

The approach I took to predict each player's strikeout percentage in the 2025 season is Bayesian Linear Regression in which we assume the strikeout percentage of a given player is a linear combination of all his given metrics, approximated using Markov Chain Monte Carlo. In other words,

$$K\% = \beta_0 + \beta_1(Stuff+) + \beta_2(Age) + \beta_3(PlayerId) + \beta_4(Team) + \beta_5(TBF)$$
$$K\% \sim Normal(\mu, \sigma)$$

Under this assumption, we can state using Bayes' Theorem that for each coefficient,

$$Posterior \ \propto Likelihood * Prior$$
$$P(\beta_i|data) \propto P(data|\beta_i) * P(\beta_i)$$

Posterior: distribution of each coefficient

Prior: initially Normal distributions, updated each iteration

Likelihood: given these coefficient distributions, how likely is the observed K %

To estimate our K%s for each line of training data (Seasons 2021-2024), we plug in our input data, sample from the coefficient distributions for each variable assuming Normal distributions for all, compare to the observed K%, update the prior, and repeat until our acceptance level has been reached.

However, there are a few problems with this approach. For one, PlayerId and Team are categorical variables. Even after we perform a simple encoding to assign each one a numerical ID for player and team, we don't want to use the value of the ID to calculate the output K%. In other words, if Jeff Hoffman is assigned ID 1 and Jhoan Duran is assigned ID 2, it doesn't mean Duran has a "better" ID like you would say a higher Stuff+ is better than a lower Stuff+. Instead, what we want to do is calculate the *effect* being a given player or on a given team has on the K%. We will call this effect variable α.

To find our α values, we build a hierarchical model. Start with creating a variable to estimate the effect of each ID, assuming a normal distribution. In other words, if two players have the same stats, but Player A has a better K% than Player B, Player A has a greater *effect*. This value is then scaled to the standard deviation of the entire population of all players to determine how significant is each player's effect in relation to all pitchers. Each iteration, the offset selects a random effect for the given player from the distribution, scales it by the standard deviation distribution, estimates the alpha value, and updates the prior. Repeat the same process with the teams.

Our updated Linear Regression model is now:

$$K\% = \beta_0 + \beta_1(Stuff+) + \beta_2(Age) + \alpha_{PlayerId} + \alpha_{Team} + \beta_5(TBF)$$
$$K\% \sim Normal(\mu, \sigma)$$

There is still one problem with this model. As TBF increases, the mean K% does not also change. The standard deviation decreases, but the mean remains the same. This can be seen in Figure 2. Thus, we should not consider K% to be a function of total batters faced. Instead, we can change the output of our model to be the number of strikeouts and run n number of trials where n is the TBF. This way, when testing on a pitcher who has faced 100 batters, we will have a more certain prediction than a pitcher who has faced 10, as opposed to having a higher K% because of a higher TBF. We do this by using a discrete Binomial distribution instead of a continuous Normal distribution. For each batter faced, the Binomial outputs either a "yes" or "no" to indicate a strikeout or not. Once we have an estimated number of strikeouts, we divide by TBF to get the predicted K%.

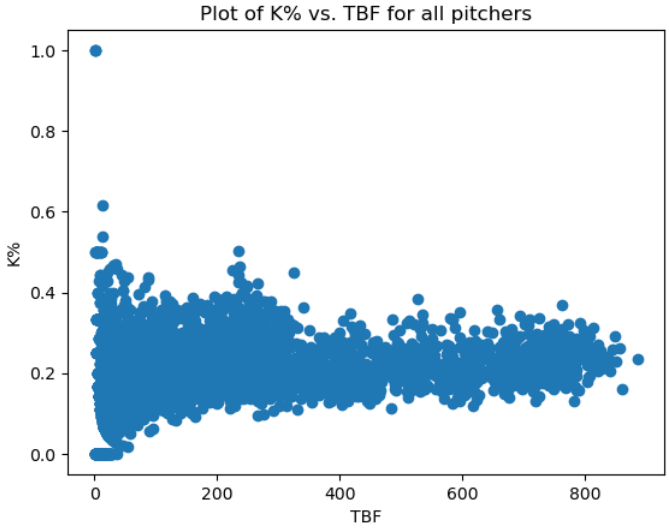


Figure 1: Graph of K% vs TBF

In order to make this change, we need to ensure the probability inputted to our Binomial model is between 0 and 1. To do so, estimate the logit function using our input data, then use the inverse logit, or sigmoid, to map all our values to probability values between 0 and 1.

Finally, our model is represented by the following:

$$
\begin{aligned}
&logit(p) = \beta_0 + \beta_1(Stuff+) + \beta_2(Age) + \alpha_{PlayerId} + \alpha_{Team} \\
&K \sim Binomial(TBF, p) \\
&K\% = K/TBF
\end{aligned}
$$

This model was created and run in Python using pandas for data reading and operations, scipy for scaling and accuracy metrics, and pymc for running the Markov Chain Monte Carlo model. The first few steps are to read in the data, replace team names ('OAK' changed to 'ATH' in 2025 due to the A's anticipated move), encode categorical data, standardize Stuff+ and Age to have mean 0 and standard deviation 1, separate training and testing data, and calculate number of strikeouts from K% to align with our new Binomial model. These steps are as follows:

```
# Load in dataset
df = pd.read_csv('k_2026.csv')

# Change all "ATH" to be "OAK" since the A's changed their name
df['Team'] = df['Team'].replace('ATH','OAK')
```

```
# Encode categorical data (PlayerId and Team)
id_factor, id_uniques = pd.factorize(df[‘PlayerId’])
df[‘PlayerId’] = id_factor
team_factor, team_uniques = pd.factorize(df[‘Team’])
df[‘Team’] = team_factor

# Standardize Stuff+ and Age
df[‘Stuff+’] = StandardScaler().fit_transform(pd.DataFrame(df[‘Stuff+’]))
df[‘Age’] = StandardScaler().fit_transform(pd.DataFrame(df[‘Age’]))

# Create training and testing datasets
input_cols = [‘PlayerId’, ‘Team’, ‘TBF’, ‘Stuff+’, ‘Age’]
output_cols = [‘K%’]
x_train = df.loc[df[‘Season’]<2025,input_cols]
y_train = df.loc[df[‘Season’]<2025,output_cols]
x_test = df.loc[df[‘Season’]==2025,input_cols]
y_test = df.loc[df[‘Season’]==2025,output_cols]

# Set input and output parameters
rng: np.random.Generator = np.random.default_rng(seed=26)
id = np.array(x_train[‘PlayerId’])
team = np.array(x_train[‘Team’])
tbf = np.array(x_train[‘TBF’])
stuff = np.array(x_train[‘Stuff+’])
age = np.array(x_train[‘Age’])
# Get number of strikeouts as opposed to strikeout percentage
y = [int(round(x)) for x in np.array(y_train).reshape(-1) * tbf]
```

The next step is to build the pymc model. We use pymc's built-in distributions for HalfNormal, Normal, and Binomial. After setting up the distributions for the coefficients, we calculate our estimated values using the logit function and compare to the observed values in the training data. This process is repeated until a 90% acceptance has been achieved. The model is seen in the code below:

```
# Make model
with pm.Model() as model:
    # Calculate the effect a given player or team has on K’s
    sigmaId = pm.HalfNormal(‘sigmaId’, 1.0)
    sigmaTeam = pm.HalfNormal(‘sigmaTeam’, 1.0)
    offsetId = pm.Normal(‘offsetId’, mu=0, sigma=1, shape=len(np.unique(id)))
    offsetTeam = pm.Normal(‘offsetTeam’, mu=0, sigma=1, shape=len(np.unique(team)))

    # Priors: Initially assume distribution for variables
    beta0 = pm.Normal(‘beta0’, mu=0, sigma=1)
    betaStuff = pm.Normal(‘betaStuff’, mu=0, sigma=1)
    betaAge = pm.Normal(‘betaAge’, mu=0, sigma=1)
    alphaId = offsetId * sigmaId
    alphaTeam = offsetTeam * sigmaTeam

    # Expected value of outcome.
    log_odds = beta0 + betaStuff*stuff + betaAge*age + alphaId[id] + alphaTeam[team]
```

```
# Likelihood (sampling distribution) of observations
p = pm.math.sigmoid(log_odds)
y_obs = pm.Binomial('y_obs', n=tbf, p=p, observed=y)

# Draw 2,000 posterior samples
trace = pm.sample(tune=1_500, draws=2_000, chains=4, cores=2, target_accept=0.9,
progressbar=True, random_seed=rng)
```

After training the model, we can visualize the distributions created using arviz. The plots in Figure 3 show the generated distributions for each coefficient, offset, and sigma value. We can see that the coefficients and sigma values were estimated from the entire population and only returned one distribution over each chain whereas the offset values have a unique distribution for each player and team. This was the expected result.

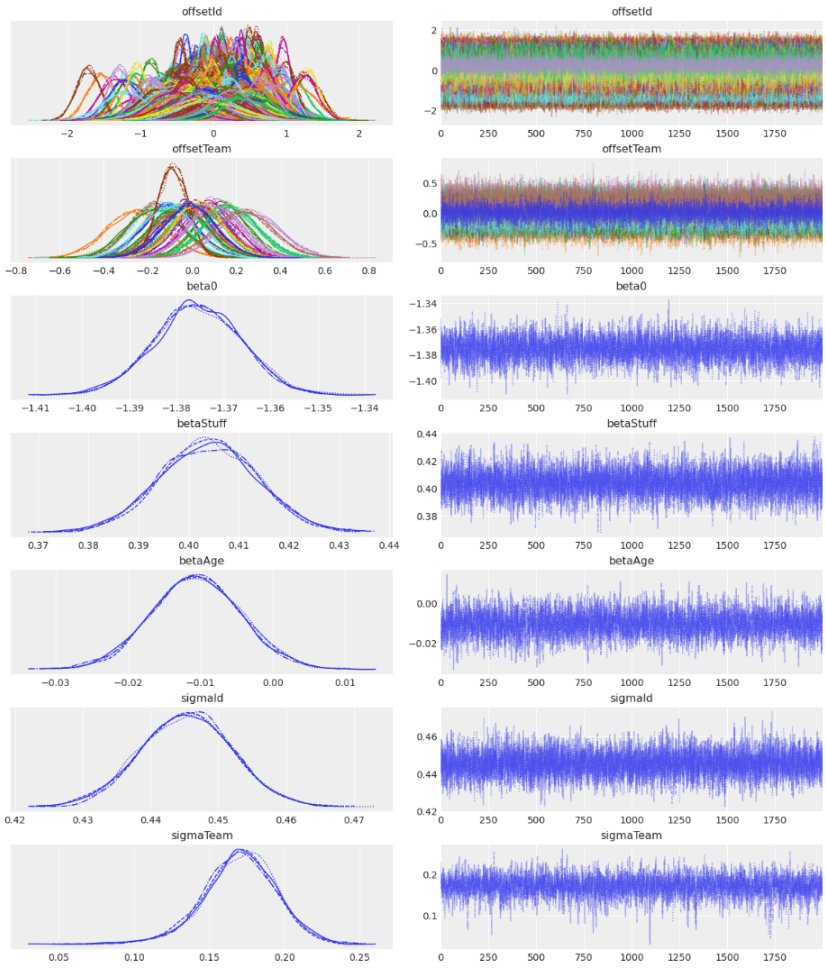


Figure 2: Graph of Generated Distributions

Finally, with our trained model, we can test the performance using our testing dataset from the 2025 Season. The steps to make our predictions are as follows:

1. For each data point, sample from distributions to estimate values for $\beta_0, \beta_1, \beta_2$, $offsetId$, $sigmaId$, $offsetTeam$, $sigmaTeam$
2. Calculate $\alpha_{PlayerId}$ and α_{Team} by multiplying offset by sigma
3. Plug in all known values and solve for $logit(p)$ using $logit(p) = \beta_0 + \beta_1(Stuff+) + \beta_2(Age) + \alpha_{PlayerId} + \alpha_{Team}$
4. Solve for p using sigmoid: $p = \frac{1}{1+exp(-logit(p))}$
5. Estimate number of strikeouts using $K = Binomial(TBF, p)$
6. Divide K by TBF to get an estimate of K%

Each step is run 100 times for each data point and the average is taken from all values to determine the estimate. The Python code to perform these steps is below:

```
# Draw samples from distributions and calculate K% for each player in testing dataset
start = time.time()
estimates = []
# Create maps for categorical data
id_map = {value: code for code, value in enumerate(id_uniques)}
team_map = {value: code for code, value in enumerate(team_uniques)}

# Get estimate for each player in 2025 data
for i in range(len(x_test)):
    guess = []
    # Draw 100 samples
    for j in range(100):
        row = x_test.iloc[i,:]

        # Draw from distributions we just made
        b0_post = trace.posterior['beta0'].values.flatten()
        b0 = b0_post[np.random.randint(len(b0_post))]
        bs_post = trace.posterior['betaStuff'].values.flatten()
        bs= bs_post[np.random.randint(len(bs_post))]
        ba_post = trace.posterior['betaAge'].values.flatten()
        ba = ba_post[np.random.randint(len(ba_post))]

        # Get Stuff+ and Age values (already scaled), PlayerId and Team values, and TBF
        stuff_new = row['Stuff+']
        age_new = row['Age']
        id_new = int(row['PlayerId'])
        team_new = int(row['Team'])
        tbf_new = row['TBF']

        # Draw from the Player offset distribution corresponding to the given player
        try:
            offsetId_post = trace.posterior.offsetId[:,:,id_new].values.flatten()
            offset_id = offsetId_post[np.random.randint(len(offsetId_post))]
            sigmaId_post = trace.posterior['sigmaId'].values.flatten()
```

```
            sigma_id = sigmaId_post[np.random.randint(len(sigmaId_post))]
            alphaId_new = offset_id * sigma_id
        except:
            # If this player was not in the training data, set their player effect to 0
            alphaId_new = 0

        # Draw from the Team offset distribution corresponding to the given team
        offsetTeam_post = trace.posterior.offsetTeam[:,:,team_new].values.flatten()
        offset_team = offsetTeam_post[np.random.randint(len(offsetTeam_post))]
        sigmaTeam_post = trace.posterior['sigmaTeam'].values.flatten()
        sigma_team = sigmaTeam_post[np.random.randint(len(sigmaTeam_post))]
        alphaTeam_new = offset_team * sigma_team

        # Solve for logit(p) value
        logit_p = b0 + bs*stuff_new + ba*age_new + alphaId_new + alphaTeam_new

        # Inverse logit(p) to get the value of p
        p = 1 / (1 + np.exp(-logit_p))

        # Find an estimate for number of K
        k = np.random.binomial(n=tbf_new, p=p)

        # Calculate K% from k and tbf
        k_percent = k/tbf_new

        guess.append(k_percent)

    estimates.append(np.mean(guess))

print(f'Time: {time.time()-start}')
```

Finally, we can use the built-in root mean square error scipy function to get an estimate of how well our model performed.

```
# Add estimates to DataFrame containing observed values
y_test['Estimate'] = estimates

# Find error
rmse = root_mean_squared_error(y_test.iloc[:,0], y_test.iloc[:,1])
print(rmse)
```

Using this Bayesian Regression model with only the given data, we were able to achieve a Root Mean Squared Error (RMSE) of 0.0591. Though small, one possible explanation of the error is the presence of position players in the data who come in during blowout games. Their data could affect the results as they may have a 0% K rate across many batters faced in a single game. To test this theory, I ran this same model while removing all data points from pitchers with 12 or less batters faced. This is just an estimate assuming a 50 pitch minimum for Stuff+ and 4 pitches per at-bat. When doing so, the model improved to a RMSE of 0.0518. Some suggested improvements made for the future would be to crosscheck the training data with player positions and only train on data from real pitchers. Further, adding another categorical variable for relief pitcher versus starting pitcher may indicate the effect of being in either of those positions.

Additionally, as with all stochastic models, tweaking of parameters through trial-and-error should naturally decrease the error.

Overall, this approach to estimating K% using Bayesian Linear Regression is well-fit because it takes population data into account (i.e. how does Stuff+ affect K% for all pitchers?) as well as individual data of each player (i.e. adjust estimates based on Player A's historical data). Likewise, this methodology produces coefficient distributions that place more value on variables that have a greater statistical significance on the output, acting as a built-in feature selection for the training data. With some small improvements, I am confident this model could be extremely effective in making predictions for K% in future seasons.

References

[1] https://pandas.pydata.org/docs/reference/api/pandas.factorize.html

[2] https://medium.com/intuition/gentle-introduction-of-bayesian-linear-regression-c83da6b0d1f7

[3] Used https://chatgpt.com/ for debugging, but code written by me

[4] https://www.pymc.io/projects/docs/en/latest/guides/Probability$_D istributions.html$